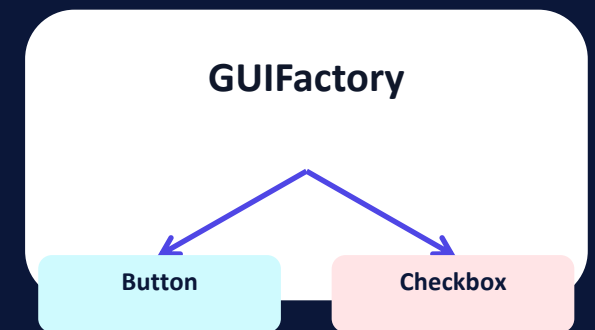


ABSTRACT FACTORY

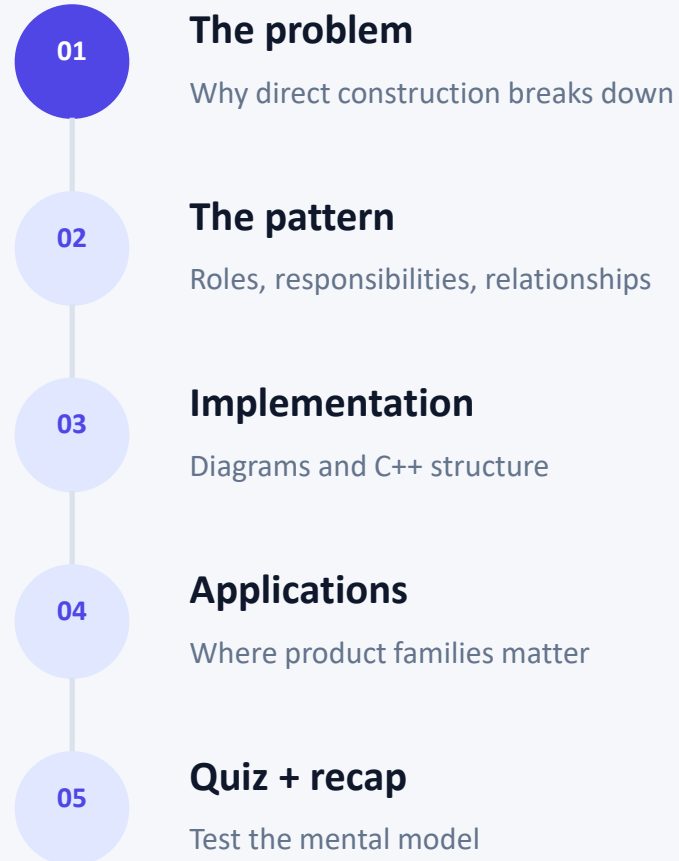
Create compatible families of objects without coupling client code to concrete classes.

ONE FACTORY

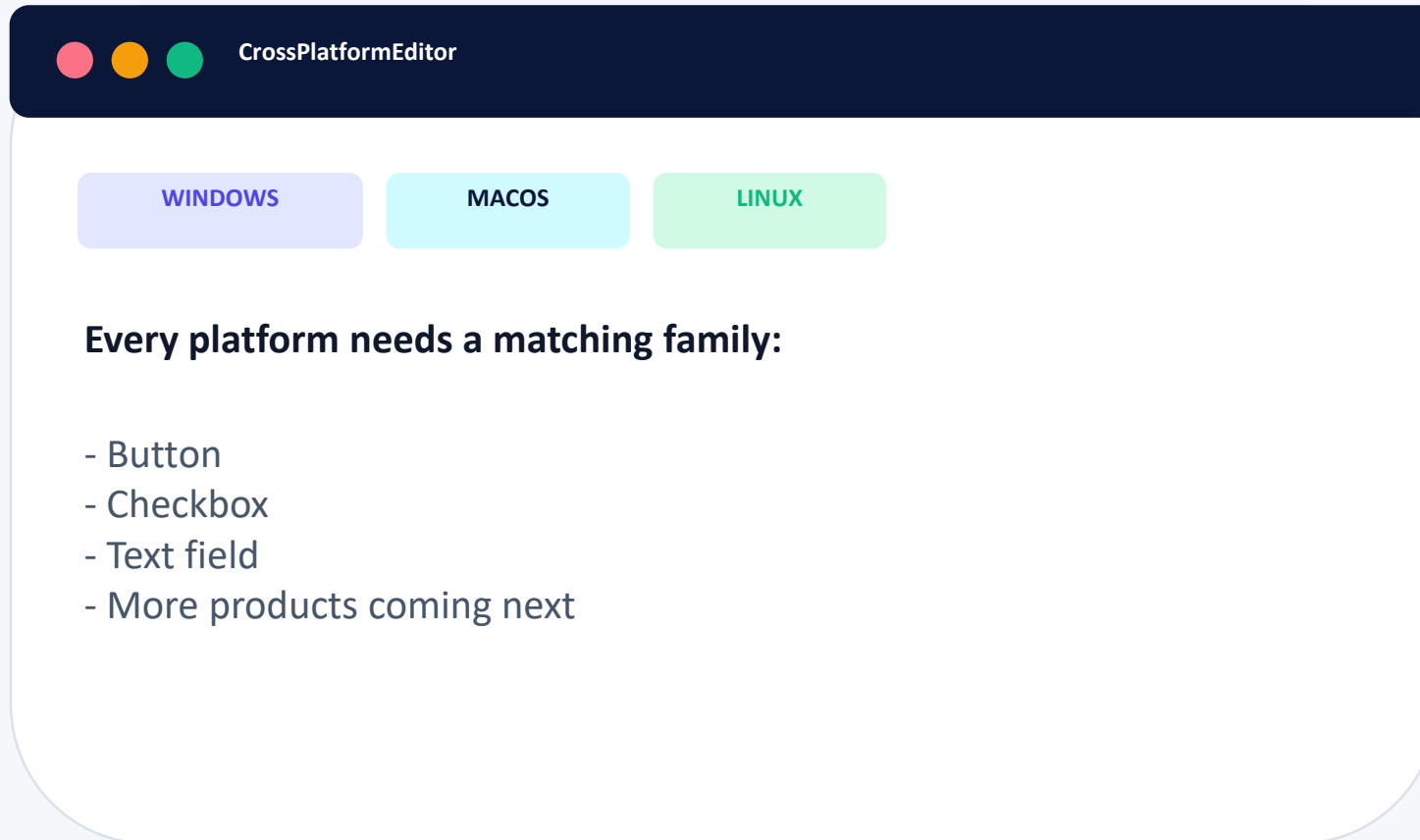


Today's route

From a brittle UI switch to a scalable family of products.



The cross-platform UI challenge



DESIGN QUESTION

**How do we create
consistent UI families
without spreading
platform checks
everywhere?**

SCALE WITHOUT REWRITES

First attempt: choose concrete classes directly

```
if (os == "Windows") {  
    button    = new WindowsButton;  
    checkbox = new WindowsCheckbox;  
}  
else if (os == "Mac") {  
    button    = new MacButton;  
    checkbox = new MacCheckbox;  
}  
...
```

NAIVE

It works - until the system grows.

Each new platform and product adds another branch, another dependency, and another place to make a mismatch.

Why the naive approach does not scale

01 Open/Closed violation

Adding a platform means editing every creation site.

02 Tight coupling

Client code knows `WindowsButton`, `MacCheckbox`, and every future concrete class.

03 Family mismatch

Nothing prevents a `Windows` button from being paired with a `macOS` checkbox.

04 Duplicated logic

The same platform switch appears wherever components are constructed.

Second attempt: factories per product

```
Button button =  
    ButtonFactory::create(os);  
  
Checkbox checkbox =  
    CheckboxFactory::create(os);
```

WE NEED ONE FACTORY FOR THE WHOLE PRODUCT FAMILY

WHAT IMPROVED

Construction moved out of the client.

WHAT REMAINS

The client still selects the OS for every product, so family consistency is not guaranteed.

The Abstract Factory pattern

Five roles separate what the client needs from how each family is built.

Abstract Factory

Declares one creation method for each product type.

Concrete Factory

Builds one complete, compatible product family.

Client

Depends only on factory and product abstractions.

Abstract Product

Defines the interface for a product category.

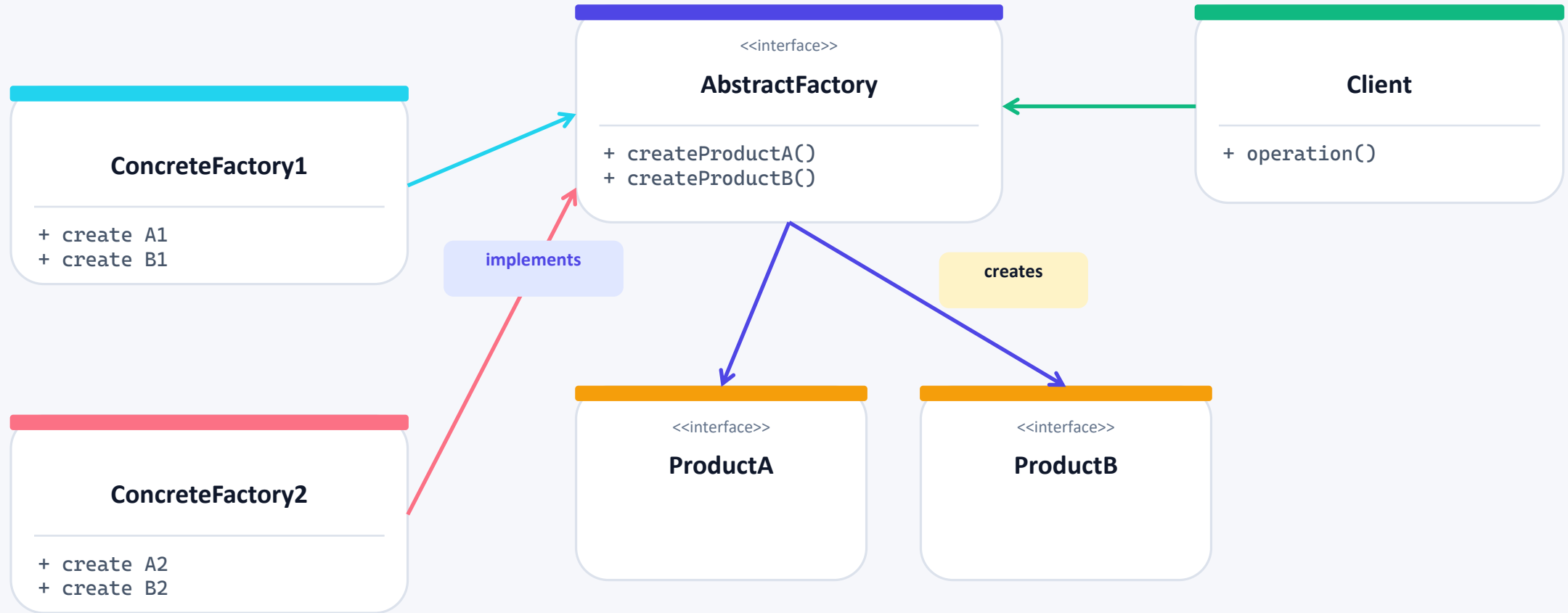
Concrete Product

Implements a product for one specific family.

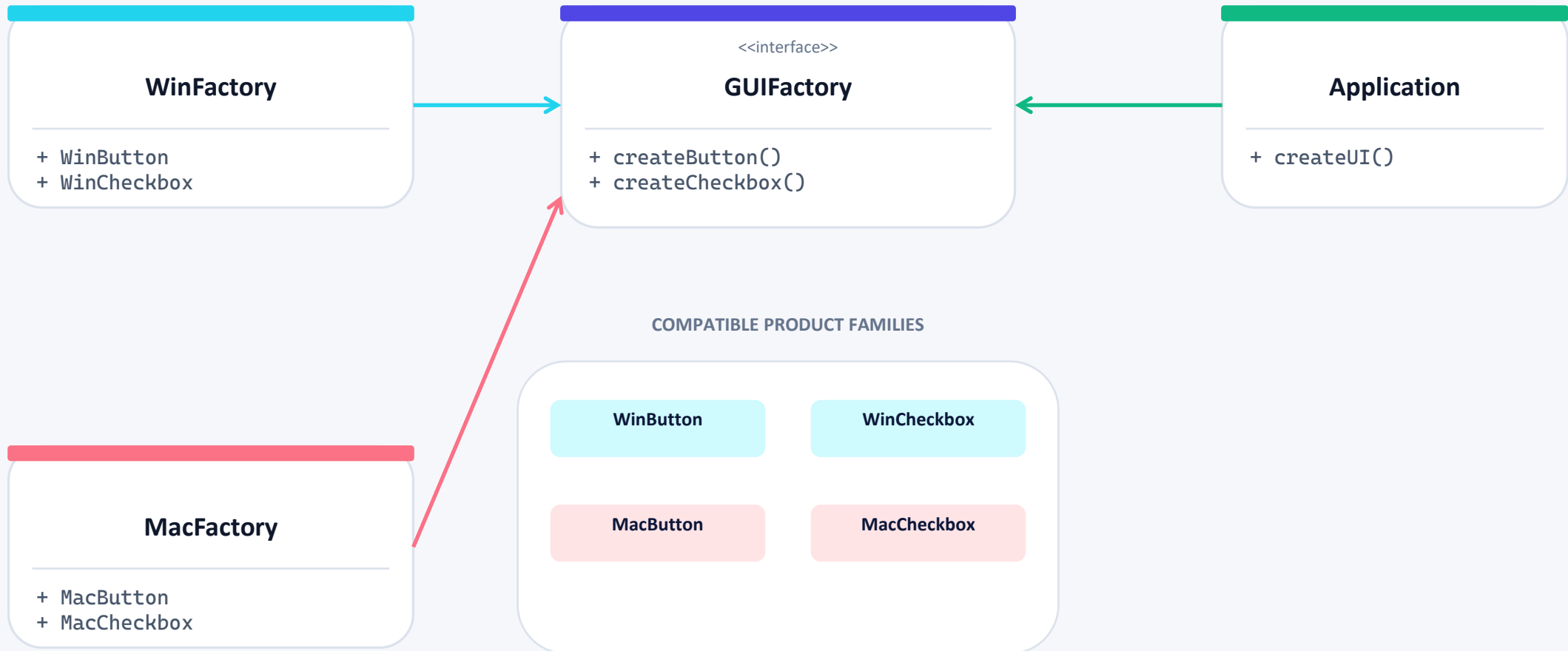
Implementation

Turn the roles into relationships, then into clean C++ interfaces.

General class relationship



Cross-platform GUI solution



C++ implementation snapshot

```
class GUIFactory {
public:
    virtual shared_ptr<Button>
        createButton() = 0;
    virtual shared_ptr<Checkbox>
        createCheckbox() = 0;
};

class WinFactory : public GUIFactory {
    // returns WinButton + WinCheckbox
};
```

CLIENT DEPENDS ON

GUIFactory
Button / Checkbox

CONCRETE FAMILY

WinFactory
WinButton / WinCheckbox

Source: abstract_factory_implementation

Trade-offs

Strong family consistency and extensibility, with an intentional increase in structure.

Where Abstract Factory earns its keep

Dependency inversion

The client depends on stable abstractions, not platform-specific classes.

Client decoupling

Concrete construction details stay behind the factory boundary.

Family consistency

Products from one factory are designed to work together.

New families are easy

Add LinuxFactory without rewriting existing client logic.

The cost of the abstraction

New product types ripple

Adding `createSlider()` changes the abstract factory and every concrete factory.

More types to navigate

Factories, interfaces, and products increase the number of classes in the system.

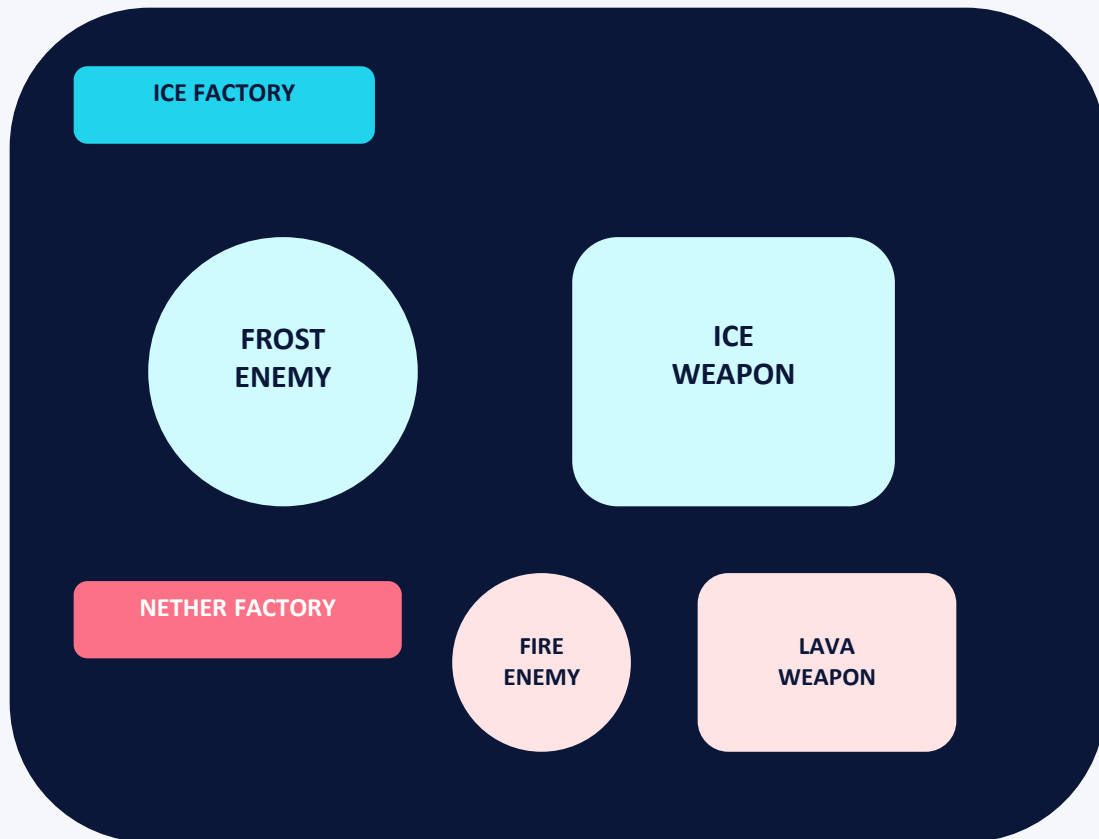
Upfront design matters

The product-family boundaries must be understood early enough to remain useful.

Applications

Use it when the system must switch complete families, not isolated objects.

Game theming: one biome, one coherent world



PATTERN MAPPING

Abstract factory

BiomeFactory

Abstract products

Enemy + Weapon

SWITCH THE FACTORY -> SWITCH THE WORLD

Knowledge check

Eight quick questions. Each answer is revealed on the following slide.

What is the main purpose of Abstract Factory?

A

Ensure a class has one instance.

B

Create families of related objects without naming concrete classes.

C

Separate complex object construction step by step.

D

Change an object's behavior at runtime.

What is the main purpose of Abstract Factory?

A

Ensure a class has one instance.

B

Create families of related objects without naming concrete classes.

CORRECT

C

Separate complex object construction step by step.

D

Change an object's behavior at runtime.

How does Abstract Factory differ from Factory Method?

A

Factory Method creates families; Abstract Factory creates one object.

B

Abstract Factory uses inheritance; Factory Method uses composition.

C

Abstract Factory creates product families; Factory Method focuses on one product type.

D

They are identical patterns.

How does Abstract Factory differ from Factory Method?

A

Factory Method creates families; Abstract Factory creates one object.

B

Abstract Factory uses inheritance; Factory Method uses composition.

C

Abstract Factory creates product families; Factory Method focuses on one product type.

CORRECT

D

They are identical patterns.

What is a major disadvantage of Abstract Factory?

A

It tightly couples clients to concrete products.

B

A new product type requires changes across the factory hierarchy.

C

Adding a new family is nearly impossible.

D

It exposes instantiation logic to the client.

What is a major disadvantage of Abstract Factory?

A

It tightly couples clients to concrete products.

B

A new product type requires changes across the factory hierarchy.

CORRECT

C

Adding a new family is nearly impossible.

D

It exposes instantiation logic to the client.

Which scenario best fits Abstract Factory?

A

A cross-platform UI toolkit with OS-specific component families.

B

A logger choosing between a file and a database.

C

A parser building a document tree step by step.

D

A cache that clones prototype objects.

Which scenario best fits Abstract Factory?

A

A cross-platform UI toolkit with OS-specific component families.

CORRECT

B

A logger choosing between a file and a database.

C

A parser building a document tree step by step.

D

A cache that clones prototype objects.

Which component actually creates the concrete objects?

A

The client

B

The abstract factory interface

C

The concrete factories

D

The abstract products

Which component actually creates the concrete objects?

A

The client

B

The abstract factory interface

C

The concrete factories

CORRECT

D

The abstract products

Which pattern is often used to manage a concrete factory instance?

A

Observer

B

Singleton

C

Strategy

D

Decorator

Which pattern is often used to manage a concrete factory instance?

A

Observer

B

Singleton

CORRECT

C

Strategy

D

Decorator

What should factory creation methods return?

A

Concrete product types

B

Abstract product interfaces

C

Generic Object or Any values

D

Boolean success flags

What should factory creation methods return?

A

Concrete product types

B

Abstract product interfaces

CORRECT

C

Generic Object or Any values

D

Boolean success flags

Which SOLID principle does Abstract Factory most directly promote?

A

Single Responsibility Principle

B

Open/Closed Principle

C

Liskov Substitution Principle

D

Dependency Inversion Principle

Which SOLID principle does Abstract Factory most directly promote?

A

Single Responsibility Principle

B

Open/Closed Principle

C

Liskov Substitution Principle

D

Dependency Inversion Principle

CORRECT

Abstract Factory vs. Factory Method

ABSTRACT FACTORY

Scope

Creates a family of related product types.

Mechanism

Composition: the client receives a factory object.

Design level

Object / family level

FACTORY METHOD

Scope

Creates one product type.

Mechanism

Inheritance: subclasses override a creation method.

Design level

Method / product level

Abstract Factory vs. Adapter

CREATIONAL

ABSTRACT FACTORY

Creates a compatible family of new objects behind shared product interfaces.

QUESTION: WHAT SHOULD WE CREATE?

STRUCTURAL

ADAPTER

Wraps an existing incompatible class so it matches the interface a client expects.

QUESTION: HOW CAN THESE CONNECT?

One factory. One compatible family.

Use Abstract Factory when clients must switch complete product families without learning concrete types.

Thank you / Questions?